

M1.1: Multi-Raft Region & Range Partitioning

Theory: Partitioning logic enabling horizontal scalability across millions of nodes. TiKV divides the global Key space into continuous 96MB slices called Regions, matching each slice to a physical Raft group.

Details: Every Store runs thousands of independent Raft consensus instances concurrently. Each instance maintains its own Peer states. Grouping data into small Regions enables sub-second rebalancing and migration across physical host nodes.

Trade-off: Managing thousands of active Raft groups creates background Heartbeat overhead. TiKV deploys a Raft Sleep mechanism to quiet inactive Regions, minimizing background CPU consumption.

M1.2: Raft Log Replication & State Machine

Theory: Ensures strong consistency and replication within a single Region's replicas.

Details: The Leader captures write requests as Raft log entries, broadcasting them to Follower nodes. Replicas write logs to their local physical 'raftdb' and reply with Acks. When the Leader receives Acks from a majority (Quorum), the log is committed and applied to the 'kvdb' state machine.

Trade-off: Serial commit logic adds latency. TiKV optimizes this using Async Commit and Pipelining, parallelizing log flushing and Ack responses across concurrent threads.

M1.3: Region Physical Split & Rebalancing

Theory: Self-healing database mechanics preventing localized hotspots and managing massive database size expansions.

Details: If a Region exceeds storage limits (e.g. 144MB) or key density overflows, the local Raft group triggers a Split operation. The Leader registers a ConfChange to divide the range into two new Regions, updating routes and notifying the PD.

Trade-off: Split operations temporarily freeze client writes on the target Region. TiKV isolates this to a metadata change and tasks the PD with background data moves.

M1.4: Region Physical Merge Sequence

Theory: Reclaims memory and disk space by clean-merging adjacent small or cold data ranges.

Details: When a Region's size drops below thresholds (e.g. 20MB), the PD schedules a Merge. Both adjacent Regions freeze writes, catch up on Raft logs to match log indices, and merge ranges.

Trade-off: Merge sequences are highly complex and prone to split-brain scenarios. TiKV restricts merges to adjacent Regions and enforces consensus log validations before committing.

M1.5: Placement Driver Global Scheduling & Routing

Theory: Distributed coordinator managing timestamps (TSO), routing tables, and replica placements.

Details: The PD uses etcd to manage cluster states. TiKV nodes report metrics (CPU, disk load, region density) via periodic heartbeats. The PD analyzes reports to schedule balancing moves.

Trade-off: The PD can become a single point of failure (SPOF) or routing bottleneck. Clients cache routing tables locally and query TiKV nodes directly, querying the PD only on cache misses.

M2.2: Lock CF Lock Management & Concurrency Conflict Control

Theory: Isolated column family dedicated to managing and tracking active Percolator transaction locks.

Details: RocksDB partitions data into Column Families (CFs). The Lock CF logs uncommitted transaction locks, mapping 'start_ts' and Primary Key locations to enable concurrent readers to detect primary commit states.

Trade-off: Frequent locking and unlocking generates numerous Tombstones in the Lock CF. TiKV runs aggressive Compaction sweeps on the Lock CF to prevent read latency degradation.

M2.1: Percolator Distributed Two-Phase Commit (2PC)

Theory: Decentralized transactional model offering row-level strong consistency without global coordinators.

Details: Clients request 'start_ts' from the PD. During Prewrite, the client locks a chosen Primary Key in the Lock CF, adding secondary locks pointing to the primary. During Commit, the client requests 'commit_ts' from the PD, unlocks the primary, writes to the Write CF, and asynchronously unlocks secondaries.

Trade-off: High contention triggers frequent prewrite conflicts, causing transaction rollbacks. TiKV mitigates this by supporting a pessimistic locking option to reserve keys.

M2.3: MVCC Physical Read/Write Pathways

Theory: Lock-free reads allowing clients to query historical data snapshots without blocking writes.

Details: Read queries carry a 'read_ts'. TiKV verifies if Lock CF contains uncommitted locks older than 'read_ts'. If clear, it queries the Write CF for the latest committed transaction at 'read_ts', locating the actual record in the Default CF via 'start_ts'.

Trade-off: Storing multiple versions causes data expansion. TiKV runs background GC threads during Compaction to reclaim space by purging versions older than the Safe Point.

M2.5: Distributed Deadlock Detection Algorithm

Theory: Identifies lock contention loops across multiple nodes in pessimistic modes.

Details: TiKV nodes maintain local Wait-For Graphs. To identify global deadlock loops, nodes send relationships to a designated coordinator node. If a loop (e.g. 'Txn A -> Txn B -> Txn C -> Txn A') is found, the target transaction is aborted.

Trade-off: Global deadlock checks add network traffic and lock latency. TiKV configures lock wait timeouts as a fallback.

M3.2: Write-Ahead Log Flushing & MemTable

Lock-Free Writes

Theory: Ensures durability and fast concurrent writes inside RocksDB.

Details: Write requests are appended to the Write-Ahead Log (WAL) on disk before being written to the MemTable. The MemTable uses a lock-free SkipList to enable parallel concurrent writes.

Trade-off: Running synchronous fsync calls on the WAL slows down throughput. TiKV runs group commits on the 'raftdb' and uses asynchronous flushing on the 'kvdb', relying on Raft consensus for durability.

M3.4: Block Cache & Bloom Filter Retrieval Acceleration

Theory: Bypasses expensive disk reads by checking key existence and caching active data blocks.

Details: RocksDB queries check files sequentially. Bloom Filters attached to SST tables run $O(1)$ verification to confirm if a key is absent, while Block Cache stores read data blocks in memory.

Trade-off: Cache structures consume massive RAM. Allocate Block Cache allocations carefully to avoid physical OOM crashes.

M2.4: Optimistic vs Pessimistic Transaction Locking

Theory: Design trade-offs balancing optimistic low-overhead executions against pessimistic conflict reductions.

Details: Optimistic transactions buffer writes on clients, acquiring locks only during the commit phase; pessimistic transactions lock keys instantly in TiKV during execution (e.g. 'SELECT FOR UPDATE').

Trade-off: Optimistic locking has zero lock overhead in clean environments but fails under high contention. Pessimistic locking adds network round-trips but prevents expensive late-stage rollbacks.

M3.1: RocksDB Dual Engine Storage Integration

Theory: Isolates disk I/O pathways by separating consensus logs from business datasets.

Details: Every TiKV node runs two RocksDB instances. The 'raftdb' stores append-only Raft logs, while the 'kvdb' stores active business data and transactional CF states (Default, Lock, Write).

Trade-off: Maintaining dual engines doubles WAL and MemTable memory allocations. TiKV optimizes the 'raftdb' for write performance, allowing users to swap it out for a custom 'Raft Engine'.

M3.3: SSTable Layering & Compaction Mechanism

Theory: Reclaims disk space and reduces read amplification by consolidating fragmented database files.

Details: MemTables flush to Level 0 SSTable files. RocksDB runs background Compaction to merge Level N files with Level N+1 files, running sort-merge steps to remove overwritten values and deleted Tombstones.

Trade-off: Compaction consumes massive disk I/O (write amplification). Under heavy writes, Compaction can starve client reads, requiring active I/O rate limiting.

M3.5: RocksDB Write Amplification Tuning

Trade-offs

Theory: Balancing write performance, read latency, and storage reuse inside LSM-Tree engines.

Details: Decreasing Compaction increases write speed but leaves deleted keys intact (space amplification) and slows reads (read amplification).

Trade-off: TiKV tunes Compaction params per CF. The Default CF uses Leveled Compaction for read speed, while the Lock CF uses aggressive settings to clear Tombstones.

M4.3: Vectorized Operator Implementation

Theory: Reduces virtual function dispatch costs and optimizes CPU caches by processing data in chunks.

Details: The Coprocessor execution engine processes records in batches (Chunks) instead of using Volcano-style row-by-row dispatches. Loops evaluate entire arrays, allowing compilers to optimize execution.

Trade-off: Vectorization requires memory buffer allocations to pass Chunks between operators, necessitating memory pools.

M5.1: gRPC Multiplexing & Heartbeat Probing

Theory: Multiplexes concurrent RPC streams over single TCP connections to reduce socket overhead.

Details: Nodes communicate via gRPC over HTTP/2, sharing connections. Built-in Keepalive frames verify physical connection health.

Trade-off: High-throughput multiplexing can trigger TCP Head-of-Line Blocking. TiKV mitigates this by maintaining multiple connections between active peers.

M5.4: Backpressure Flow Limitation Self-Healing

Theory: Protects storage nodes under heavy load by slowing down write confirmations.

Details: If RocksDB Compaction falls behind and Level 0 files exceed thresholds (e.g. 30), TiKV delays Raft Acks and gRPC reads. This slows down client writes and protects the node from OOM.

Trade-off: Triggers write latency spikes on clients but prevents nodes from crashing.

M4.1: Coprocessor Query Pushdown Execution

Theory: Decreases network latency by running filtering and aggregation calculations directly on storage nodes.

Details: Instead of sending raw rows over the network, SQL aggregates (e.g. ‘SELECT COUNT(*) WHERE age > 18’) are pushed to TiKV’s ‘Coprocessor’, which runs filter logic on local tables and returns only computed results.

Trade-off: Running calculations on storage nodes consumes CPU cycles. Poorly optimized queries can starve Raft heartbeats, requiring CPU priority scheduling.

M4.4: Chunk Memory Compact Layout & Flowing

Theory: Column-oriented memory buffer structure optimizing serialization and network transfers.

Details: Chunks group values contiguously in column arrays. When sending data over gRPC, data is copied directly from column buffers to socket streams without format translation.

Trade-off: Requires structured layouts. Supporting null values requires allocating bitmap indexes, which adds minor complexity.

M5.2: Raft Message Flow Asynchronous Pipeline Processing

Theory: Increases performance by pipelining networking, log writes, and state machine updates.

Details: The Raftstore splits message handling into three pipeline steps: Network Read Log Write (raftdb) Apply (kvdb). Steps run on separate threads connected by lock-free ring buffers.

Trade-off: Pipelining can cause queue buildup under heavy loads. If disks write slowly, Apply threads starve, requiring ingress rate limits.

M6.1: TiKV Network Partition Fault Self-Healing

Theory: Protects database consistency against network splits and brain splits.

Details: During network partitions, the majority partition (Quorum) continues processing writes while the minority partition stalls. Replicas catch up and align logs when connections recover.

Trade-off: Minority partitions can serve stale data. Enable Lease Reads or Read Index options to verify leader lease validity.

M4.2: DAG Request Parsing Engine

Theory: Rebuilds execution query plans from incoming RPC protobuf data inside storage nodes.

Details: The Coprocessor parses incoming DAG structures, rebuilding operator trees (e.g. TableScan -> Selection -> HashAgg). It maps query columns to local RocksDB layouts and instantiates iterator chains.

Trade-off: Parsing plans on the fly adds latency. TiKV caches execution contexts to avoid parsing overhead.

M4.5: CPU SIMD Instructions Compilation Acceleration

Theory: Leverages host hardware architectures to run parallel computations across data chunks.

Details: Compiling TiKV with target extensions (e.g. AVX-512, Neon) enables the Rust compiler to translate chunk-based loops into parallel SIMD instructions.

Trade-off: SIMD binaries are tied to host CPU models. Running them on older platforms triggers SIGILL crashes.

M5.3: Node-to-Node Zero-Copy Snapshot Synchronization

Theory: Transfers large state files directly from disk to network interfaces without CPU copying.

Details: When replicas catch up, the Leader generates SST snapshots. The file is sent via ‘sendfile’ directly from the OS page cache to network sockets, bypassing user-space memory.

Trade-off: Scanning databases to generate SST files creates temporary I/O spikes. TiKV rate-limits snapshot bandwidth to protect writes.

M6.2: Failpoint Injection & Chaos Engineering

Theory: Verifies database resilience under extreme failures by injecting mock errors during development.

Details: TiKV embeds the ‘fail-rs’ framework. Mock injection points are placed inside critical paths (e.g. before write, during network send). Test suites trigger these mock errors to verify recovery paths.

Trade-off: Injection code is removed during release builds via conditional compilation, ensuring zero overhead in production.

M6.3: RocksDB Runtime Monitoring Metrics

Theory: Exposes internal engine performance and health statistics.

Details: Captures and exposes engine metrics: SST file count, write stalls, MemTable sizes, block cache hits, and compaction throughput. Data is polled by Prometheus and visualized in Grafana.

Trade-off: Reading metrics synchronously can create lock contention. TiKV polls metrics asynchronously at fixed intervals.

M6.4: Distributed Tracing Trace Diagnostics

Theory: Tracks the lifecycle of transactional queries across RPC boundaries and engine modules.

Details: TiKV integrates 'minitrace'. Transactions carry a Trace ID across RPC calls, logging execution span details (e.g. 'raftstore_commit_time') for diagnostic reporting.

Trade-off: Full tracing consumes disk space and network bandwidth. Enable tracing selectively or only for slow queries in production.

Zone T1: TiKV Crate APIs

```
tikv::Storage::async_write(), raftstore::store::RaftCmd(),
coprocessor::Endpoint::handle_request(), txn::Prewrite(),
txn::Commit(), rocksdb::DB::write_cf(), rocksdb::DB::get_cf()
```

Zone T2: System Data Faults

```
percolator_lock_conflict, write_stall_backpressure,
raft_log_lag_desync, network_partition_split_brain,
rocksdb_compaction_io_saturation, distributed_deadlock_detected
```